

Classification and Regression Using Supervised Learning

DR. ULLA DELFANA ROSIANI, ST, MT

And ROKHIMATUL WAKHIDAH, S.Pd., M.T.

(TEAM TEACHING ARTIFICIAL INTELLIGENCE D4 IT)

Outline

- Supervised vs Unsupervised learning
- Supervised Learning
- Supervised Learning Classification
- Supervised Learning Regression
- Dataset, Evaluation Model, Challenge And Case Study

Supervised vs Unsupervised Learning



Supervised vs Unsupervised Learning

DEFINITION

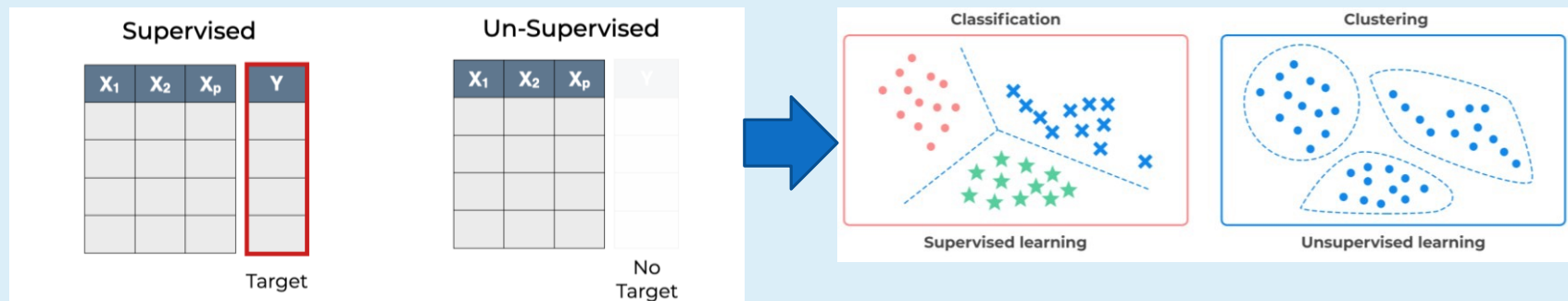
Supervised Learning is method in where model trained with data labeled, that is data which has input (feature) And output (label).

Objective: Predicting or classify data based on pattern from data train.

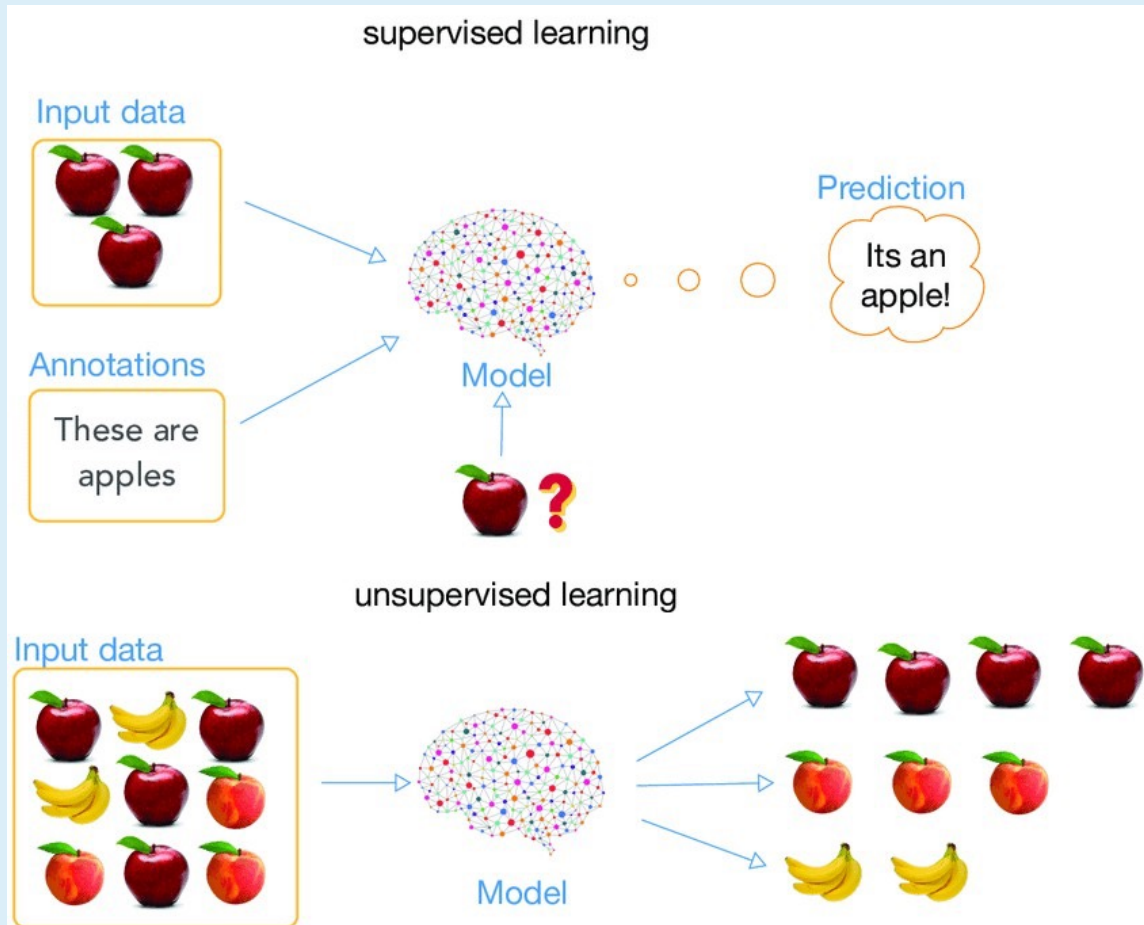
Whereas

Unsupervised Learning is method in where model trained with data which unlabeled, only use feature without known output.

Objective: Find pattern or structure hidden in data.



Supervised vs Unsupervised Learning



Supervised vs Unsupervised Learning

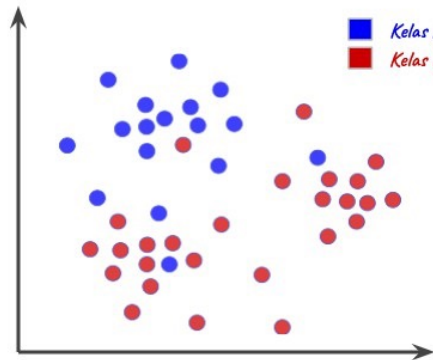
Aspect	Supervised Learning	Unsupervised Learning
Definition	Model trained with data labeled, data has input (feature) and output (label).	Model trained with unlabeled data, just use features without known output.
Objective	Predicting or classify data based on pattern from data train.	Find pattern or hidden structure in data.
Type Problem	Classification (classification) and Regression (regression).	Clustering (grouping) and Dimensionality Reduction.
Algorithm Example	• Linear Regression • Decision Tree • Random Forest • Support Vector Machine (SVM) • Neural Networks	• K- Means Clustering • Hierarchical Clustering • Principal Component Analysis (PCA) • Autoencoders

Supervised vs Unsupervised Learning

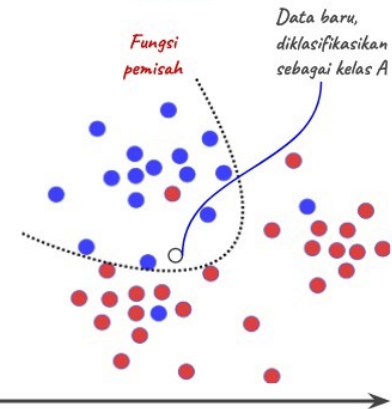
Supervised Learning

Data training sudah diketahui label/kelasnya. Training bertujuan mencari garis/fungsi pemisah antara data-data dengan label berbeda.

Data Training

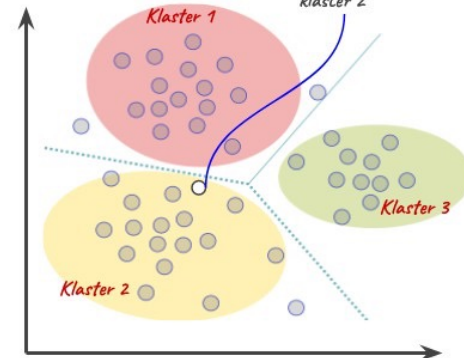
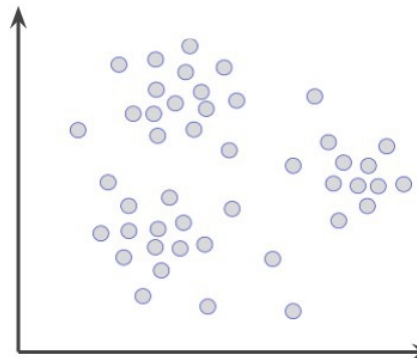


Model



Unsupervised Learning

Data training tidak diketahui kelas/labelnya. Training bertujuan untuk mengelompokkan data ke dalam kluster-kluster.



Supervised Learning



Understanding Supervised Learning

- **Supervised learning** is machine learning method where machine study from data along with the expected results, the answer, or the label. For example, so that program can recognize picture animal, program will "trained" with a bunch animal pictures which has paired or given name. Program trained for distinguish email rubbish from Which No.
- Because supervised learning is faster, cheaper, and easy to be measured, supervised learning is a popular method and most often used.
- There is two type problem which can be resolved use supervised learning that is:
 1. **Classification**
 2. **Regression**

In classification, which predicts category or class, for example spam email classification, prediction of whether a student will pass or fail (differentiate from regression, which predicts what the student's final grade will be), prediction of the rise and fall of the Rupiah exchange rate, and so on.

In regression, the predictions produce continuous numeric values. For example, regression cases are predicting a student's final grade, predicting the rupiah exchange rate, predicting stock prices, predicting rainfall, and so on.

Difference *Classification* And *Regression*

Kategori	Classification	Regression
Output	Kategori atau label	Nilai kontinu
Contoh	Spam atau tidak spam	Harga rumah
Algoritma	Decision Tree, SVM, KNN, Naïve Bayes	Linear Regression, Random Forest, SVR



Supervised Learning Demerits

Supervised Learning has a number of demerits, such as:

1. Requires large amount of Labeled Data

- Process of data labelling can be expensive and time consuming
- If the label is not available or not accurate, model can experience decline performance.

2. Vulnerable to Overfitting

- Models can become too complex and overfit to the training data.
- As a result, the model performs very well on the training data but poorly on new data.

3. Limited to Learned Patterns

- The model can only predict based on patterns that have been learned.
- If there is a change in the data pattern (e.g. a new trend), the model may not be able to adapt well.

4. Less Effective for Unstructured Data

- Complex data such as images, videos, or text often require additional techniques such as deep learning.
- This process can be more complicated than unsupervised learning methods such as clustering or autoencoders.

5. Can Suffer from Model Bias

- If the training data is not representative, the model can produce biased or inaccurate predictions.
- Example: If the dataset only contains images of dogs but no cats, the model will have difficulty recognizing cats.

6. Difficult to Use for Problems with Many Classes or Labels

- In the case of multi-class or multi-label classification, supervised learning can be more complex and require special techniques such as One-vs-All or One-vs-One.

7. Can be Computationally Expensive

- Supervised learning algorithms, especially deep learning, often require large computational power and long training times.

Supervised Learning- Classification



Classification - Definition

- Classification is a supervised learning technique that aims to categorize data into certain classes.
- Examples: Spam email detection, disease diagnosis, facial recognition.

Classification Types

1. Binary Classification → Two classes (example: spam email or not spam).
2. Multiclass Classification → More than two classes (example: flower type classification).
3. Multilabel Classification → Each sample can have more than one label.

Multi-label vs Multiclass Difference Classification

Multilabel Classification is a type of classification where each data sample can have more than one label at a time. Unlike **Multiclass Classification**, where each sample only has one label from several available categories, **Multilabel Classification** allows one sample to belong to several categories simultaneously.

Aspect	Multilabel Classification	Multiclass Classification
Number of Labels per Sample	Can be more than one	Only one
Sample Dataset	Articles can be in the categories "Technology" and "Health" at the same time	Articles can only be in one category, e.g. "Technology" only
Model Output	Binary vector with multiple values 1 (e.g. [1, 0, 1] means it belongs to both the 1st and 3rd classes)	Only one class is selected (e.g. 2, means only the 2nd class is selected)

Classification Algorithm

1. K- Nearest Neighbors (KNN)
2. Naive Bayes
3. Support Vector Machine (SVM)
4. Decision Tree
5. Random Forest

Algorithm *Classification*

1. K- Nearest Neighbors (KNN)

Concept:

Instance-based method that searches for K nearest neighbors to determine the class of a new data.

The class that appears most frequently among those neighbors will be the predicted result.

It is also called a **lazy learner algorithm** because it does not learn from the training set directly but rather stores the dataset and at the time of classification, performs actions on the dataset.

◆ Advantages:

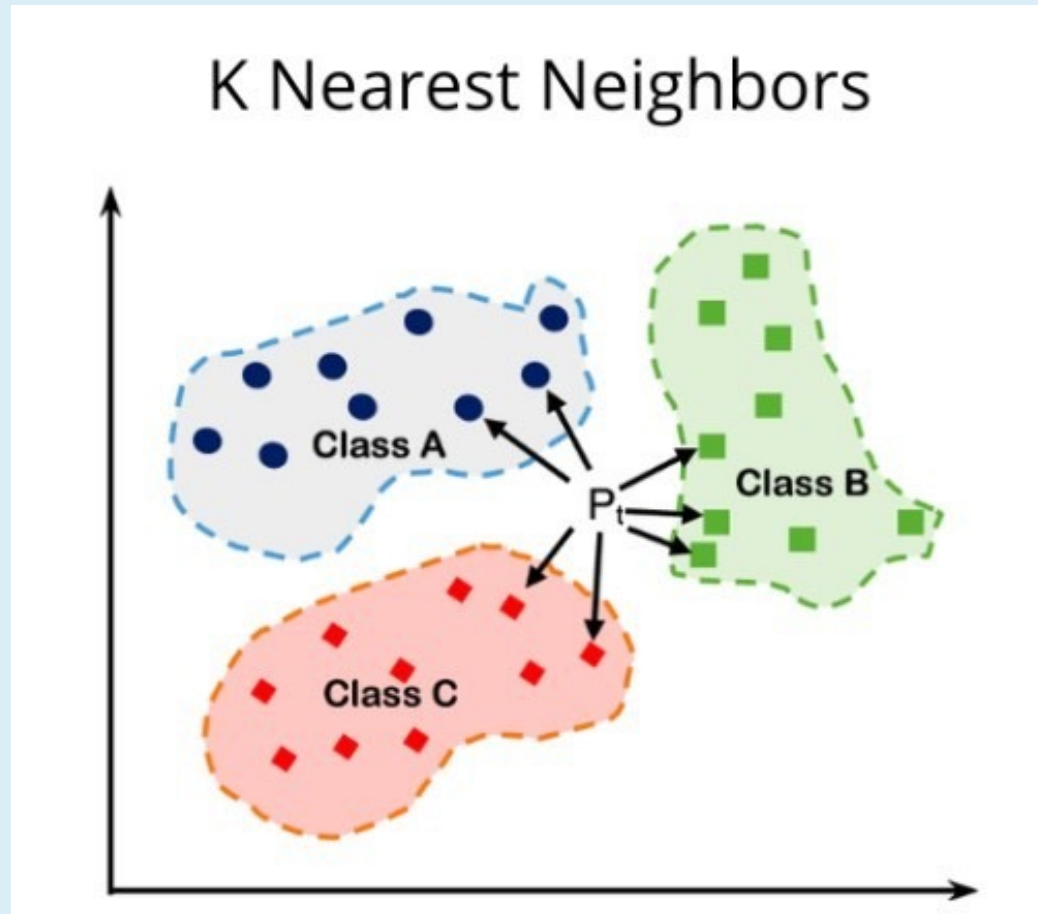
- ✓ Easy to understand and implement.
- ✓ Does not require model training, just stores the training data.

◆ Disadvantages:

- ✗ Slow for large datasets because it needs to calculate the distance to all training data.
- ✗ Vulnerable to outliers and irrelevant features.

Classification Algorithm

1. K- Nearest Neighbors (KNN)



Classification Algorithm

1. K- Nearest Neighbors (KNN)

Simple Case Example:

We have the following dataset that shows the relationship between **weight** (kg) and **height** (cm) to determine whether a person is male (M) or female (F).

Weight (kg)	Height (cm)	Class
55	160	P
58	162	P
60	165	P
63	170	L
65	172	L
70	175	L

Question:

If there is a person weighing 61 kg and 167 cm tall, is this person male or female? Use $K = 3$ to determine the answer!

Classification Algorithm

1. K- Nearest Neighbors (KNN)

KNN Calculation Steps:

1. Count Euclidean Distance

We use formula **Euclidean Distance** to measure distance between the new data new and every data in dataset:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

dengan:

- x_1, y_1 → koordinat dari titik data (berat, tinggi).
- x_2, y_2 → koordinat dari data baru (61 kg, 167 cm).

Count distance For every point:

Weight (kg)	Height (cm)	Class	Distance to (61,167)
55	160	P	$\sqrt{(61 - 55)^2 + (167 - 160)^2} = \sqrt{6^2 + 7^2} = \sqrt{36 + 49} = 9.22$
58	162	P	5.83
60	165	P	2.24
63	170	L	3.61
65	172	L	6.40
70	175	L	12.04

Classification Algorithm

1. K- Nearest Neighbors (KNN)

KNN Calculation Steps:

2. Choose K Nearest Neighbor (K = 3)

We sort distance from the smallest:

Heavy (kg)	Tall (cm)	Class	Distance
60	165	P	2.24
63	170	L	3.61
58	162	P	5.83
65	172	L	6.40
55	160	P	9.22
70	175	L	12.04

3 Nearest Neighbors :

1. P (2.24)

2. L (3.61)

3. P (5.83)

3. Determine Class with Majority

From **3 neighbor closest** , we own:

- **2 P (Female)**

- **1 L (Man)**

Because majority is **Woman (P)** , so data new **classified as Woman** .

Classification Algorithm

1. K- Nearest Neighbors (KNN)

In K-Nearest Neighbors (KNN) method, **measurement distance** very important for determine proximity between data test with data training. Some methods general For measure distance in KNN is:

- ← **1. Euclidean Distance** → Suitable for numeric data with a uniform scale.

Formula:

$$d(A, B) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

- 2. Manhattan Distance** → Suitable If difference the scale between feature is large.

Formula:

$$d(A, B) = \sum_{i=1}^n |A_i - B_i|$$

- 3. Minkowski Distance** → Flexible with parameter p .

Formula:

$$d(A, B) = \left(\sum_{i=1}^n |A_i - B_i|^p \right)^{\frac{1}{p}}$$

- 4. Chebyshev Distance** → Suitable for maximum boundary

cases. Formula: $d(A, B) = \max_i |A_i - B_i|$

- 5. Cosine Similarities** → Suitable for text or vector data where direction is more important than absolute distance. Formula:

$$\cos(\theta) = \frac{A \cdot B}{||A|| \cdot ||B||}$$

Classification Algorithm

1. K- Nearest Neighbors (KNN)

Example Implementation with Python:

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

# Data training: Heavy (kg), Tall (cm)
X_train = np.array([[55, 160], [58, 162], [60, 165], [63, 170], [65, 172],
[70, 175]])
y_train = np.array(["P", "P", "P", "L", "L", "L"]) # Label Class

# Data new Which want to classified
X_new = np.array([[61, 167]])

# Make model KNN with K = 3
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

# Prediction class For data new
y_pred = knn.predict(X_new)
print(f"Data {X_new.tolist()} classified as: {y_pred[0]}")
```

Classification Algorithm

2. Naive Bayes

◆ Concept:

Based on Bayes' Theorem, calculating the probability of a class based on the probability distribution of features.

It is assumed that each feature is independent (Naïve Assumption).

The way the Naive Bayes algorithm works is governed by Bayes' theorem which works according to the principle of conditional probability. Conditional probability represents a probability or chance of an event, which refers to related events that have occurred.

◆ Advantages:

✓ Fast and efficient, even for large datasets.

✓ Suitable for text classification such as Spam Detection and Sentiment Analysis.

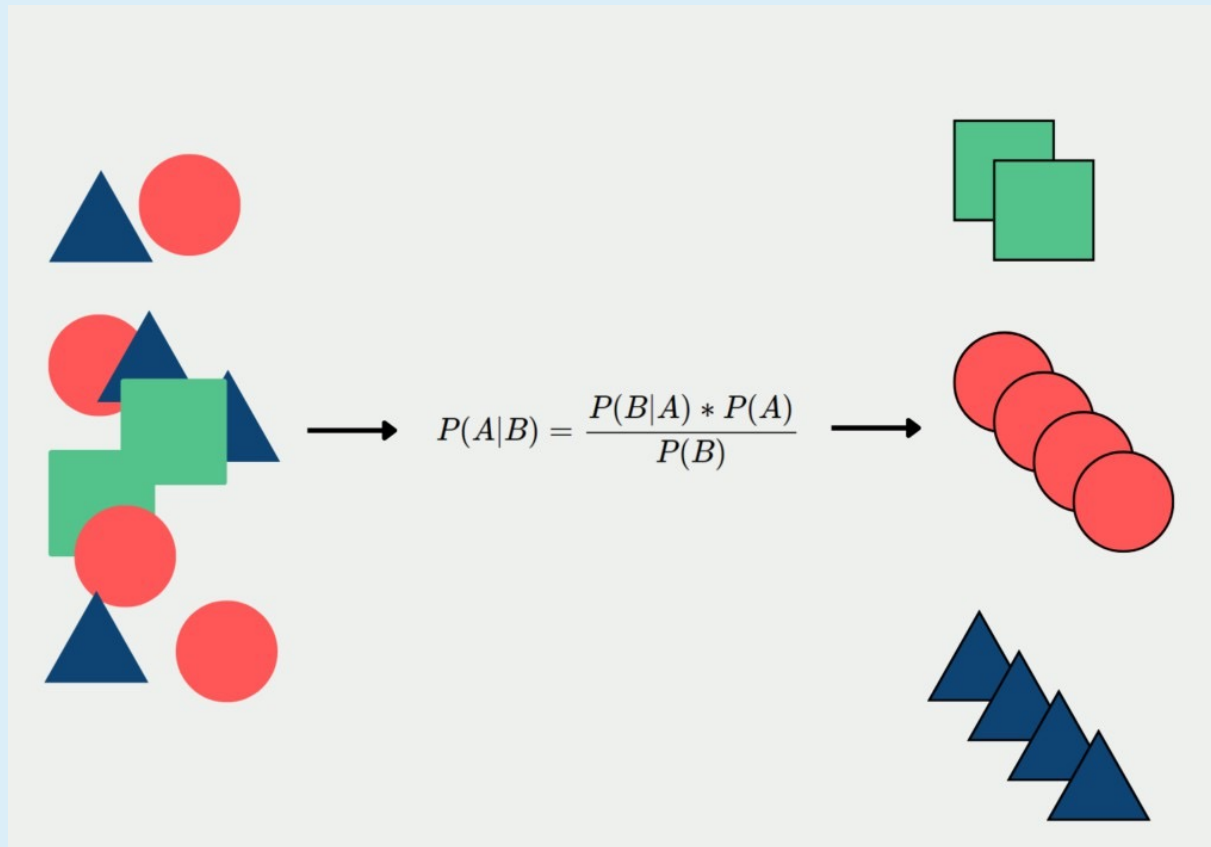
◆ Disadvantages:

✗ The assumption of feature independence is rarely true in the real world.

✗ Not suitable for data with highly correlated features

Classification Algorithm

2. Naive Bayes



Classification Algorithm

2. Naive Bayes

Example Implementation with Python:

```
from sklearn.naive_bayes import GaussianNB

# Initialization model Naive Bayes
nb = GaussianNB()

# Train the model
nb.fit(X_train, y_train)

# Prediction
y_pred = nb.predict(X_test)
```

Classification Algorithm

3. Support Vector Machine (SVM)

◆ Concept:

Support Vector Machine (SVM) is a learning algorithm used for classification and regression. Its goal is to find the best line or hyperplane that separates two classes of data with the maximum margin between the classes. Margin is the distance between the line/hyperplane and the nearest points of each class. SVM looks for important points called support vectors that are located around the boundary line/hyperplane.

Finds the **optimal hyperplane** that separates the classes with the largest margin.

Uses **kernel trick** to handle data that cannot be separated linearly.

- **Advantages**

good for high-dimensional data

suitable for non-normally distributed data

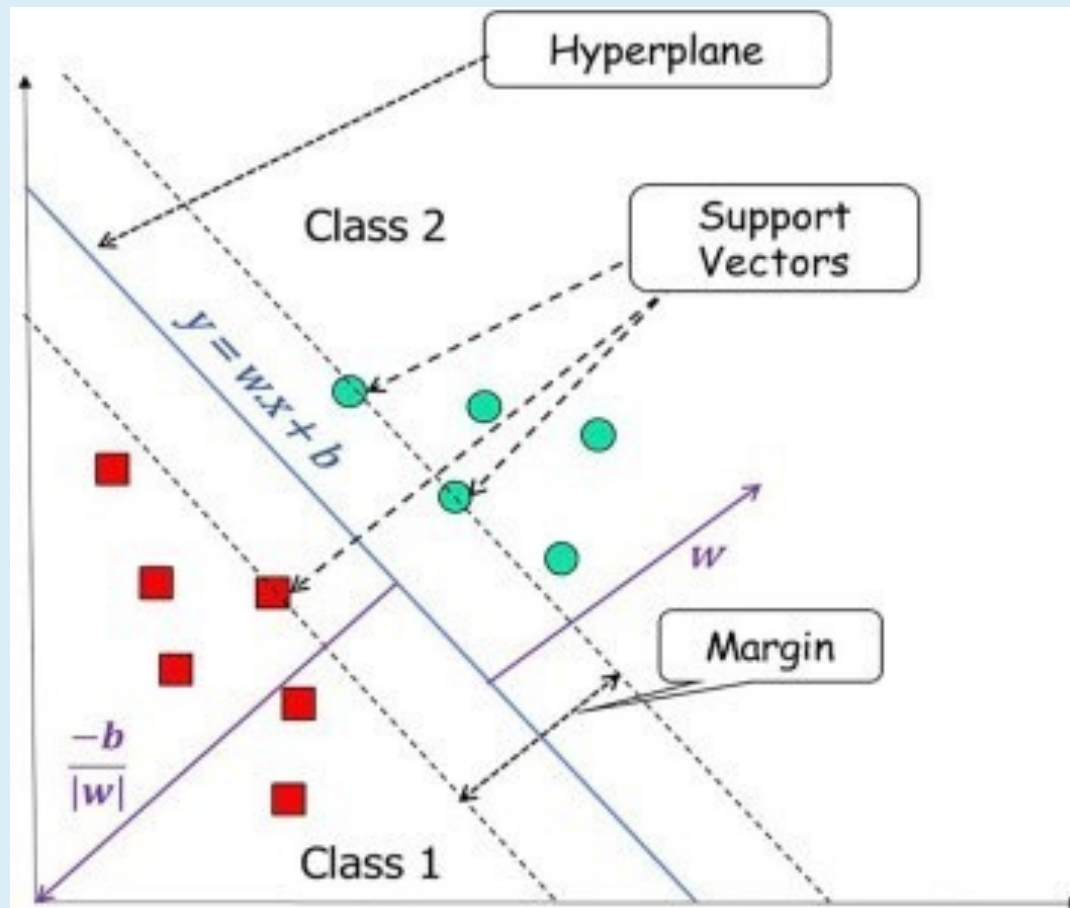
- **Disadvantages**

slow if the dataset is very large

difficult to choose the right kernel

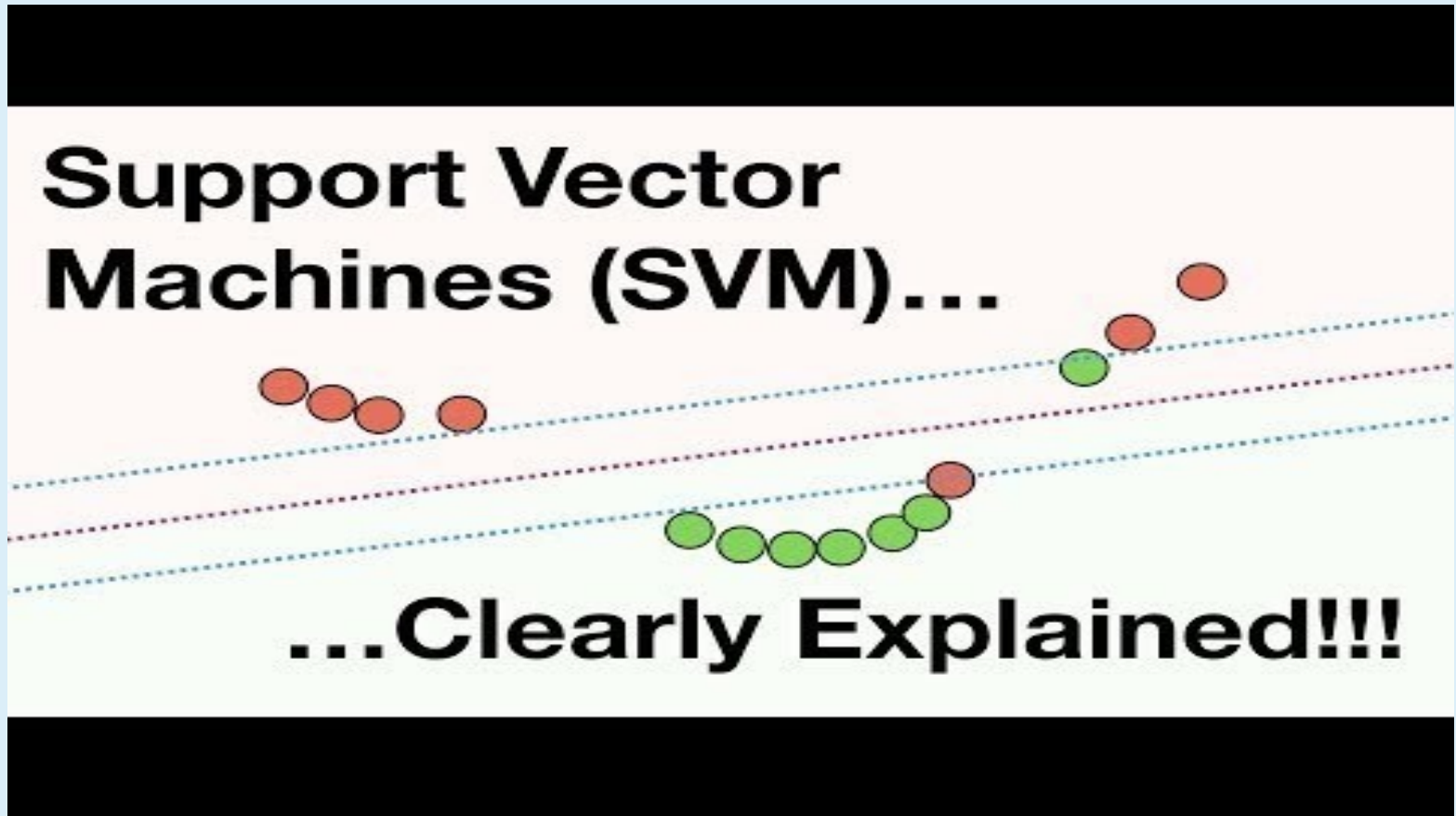
Classification Algorithm

3. Support Vector Machine (SVM)



Classification Algorithm

3. Support Vector Machine (SVM)



Source video: <https://youtu.be/efR1C6CvhmE?feature=shared>

Classification Algorithm

3. Support Vector Machine (SVM)

In **Support Vector Machine (SVM)**, A kernel is a function that transforms data into a higher dimension to make **it easier to separate linearly**. Kernels are used to handle data that **is not linearly separable** in the original space. Here are some types of kernels commonly used in SVM:

Type Kernel	When Used?	Excess	Lack
Linear	Data can separated linearly	Fast, simple, no additional parameters needed	Cannot handle non-linear data
Polynomial	Non- linear Data with polynomial pattern	Can handle some of non-linearities	Can be overfitting if the degree is high.
RBF (Gaussian)	Complex and non-linear data	Flexible, most often used.	The selection parameter γ is important.
Sigmoid	If you want to approach Neural Network model.	Can handle Some of of non-linear cases	Not enough stable compared to RBF.

Classification Algorithm

3. Support Vector Machine (SVM)

Popular kernels used with SVMs include:

Type of SVM	Mercer Kernel	Description
Gaussian or Radial Basis Function (RBF)	$K(x_1, x_2) = \exp\left(-\frac{\ x_1 - x_2\ ^2}{2\sigma^2}\right)$	One class learning. σ is the width of the kernel
Linear	$K(x_1, x_2) = x_1^T x_2$	Two class learning.
Polynomial	$K(x_1, x_2) = (x_1^T x_2 + 1)^\rho$	ρ is the order of the polynomial
Sigmoid	$K(x_1, x_2) = \tanh(\beta_0 x_1^T x_2 + \beta_1)$	It is a mercer kernel for certain β_0 and β_1 values only www.softscients.web.id

Classification Algorithm

3. Support Vector Machine (SVM)

Example Implementation with Python:

```
from sklearn.svm import SVC

# Initialization model SVM with kernel linear
svm = SVC(kernel='linear')

# Train the model
svm.fit(X_train, y_train)

# Prediction
y_pred = svm.predict(X_test)
```


Classification Algorithm

4. Decision Tree

◆ Concept:

Building a decision tree, where each node represents a feature, each branch represents a decision, and each leaf is a classification result.

This method is called a 'tree' because its shape resembles a tree with many branches. In a decision tree, you can arrange various choices and investigate the possible outcomes of those choices.

◆ Advantages:

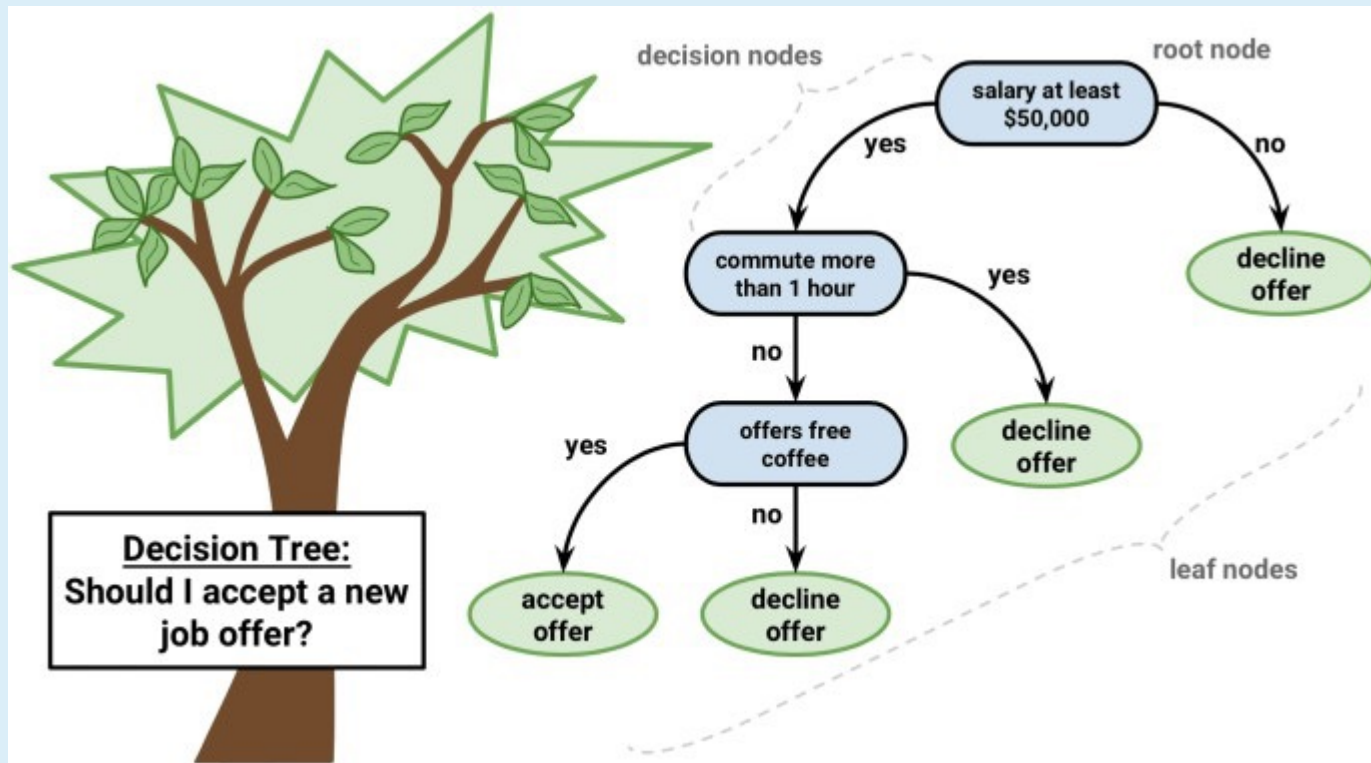
- ✓ Easy to understand and visualize.
- ✓ Does not require data normalization.

◆ Disadvantages:

- ✗ Susceptible to overfitting, especially if the tree is too deep.
- ✗ Not suitable for data with many imbalanced classes.

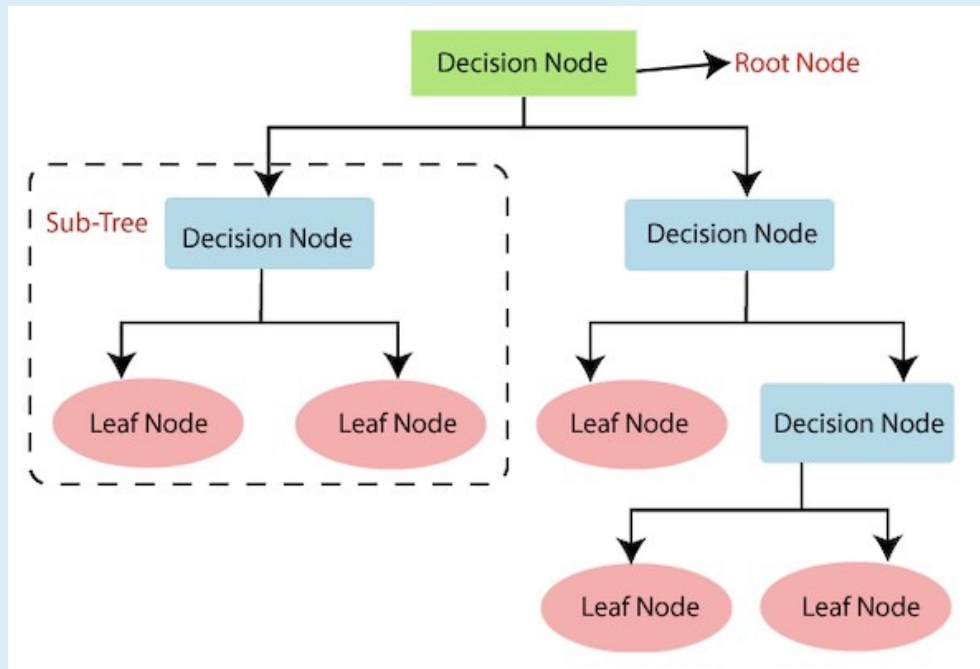
Classification Algorithm

4. Decision Tree



Classification Algorithm

4. Decision Tree



There are three elements in a decision tree, namely:

- root node (root): the final goal or major decision to be taken
- branches (twigs): various choices of action
- leaf node (leaf): possible outcomes of each action

Classification Algorithm

4. Decision Tree

Example Implementation with Python:

```
from sklearn.tree import DecisionTreeClassifier

# Initialization model Decision Tree
dt = DecisionTreeClassifier()

# Train the model
dt.fit(X_train, y_train)

# Prediction
y_pred = dt.predict(X_test)
```

Classification Algorithm

5. Random Forest

◆ Concept:

A combination of many Decision Trees working together (ensemble learning).

Each tree is trained with a different subset of the data to reduce overfitting.

◆ Advantages:

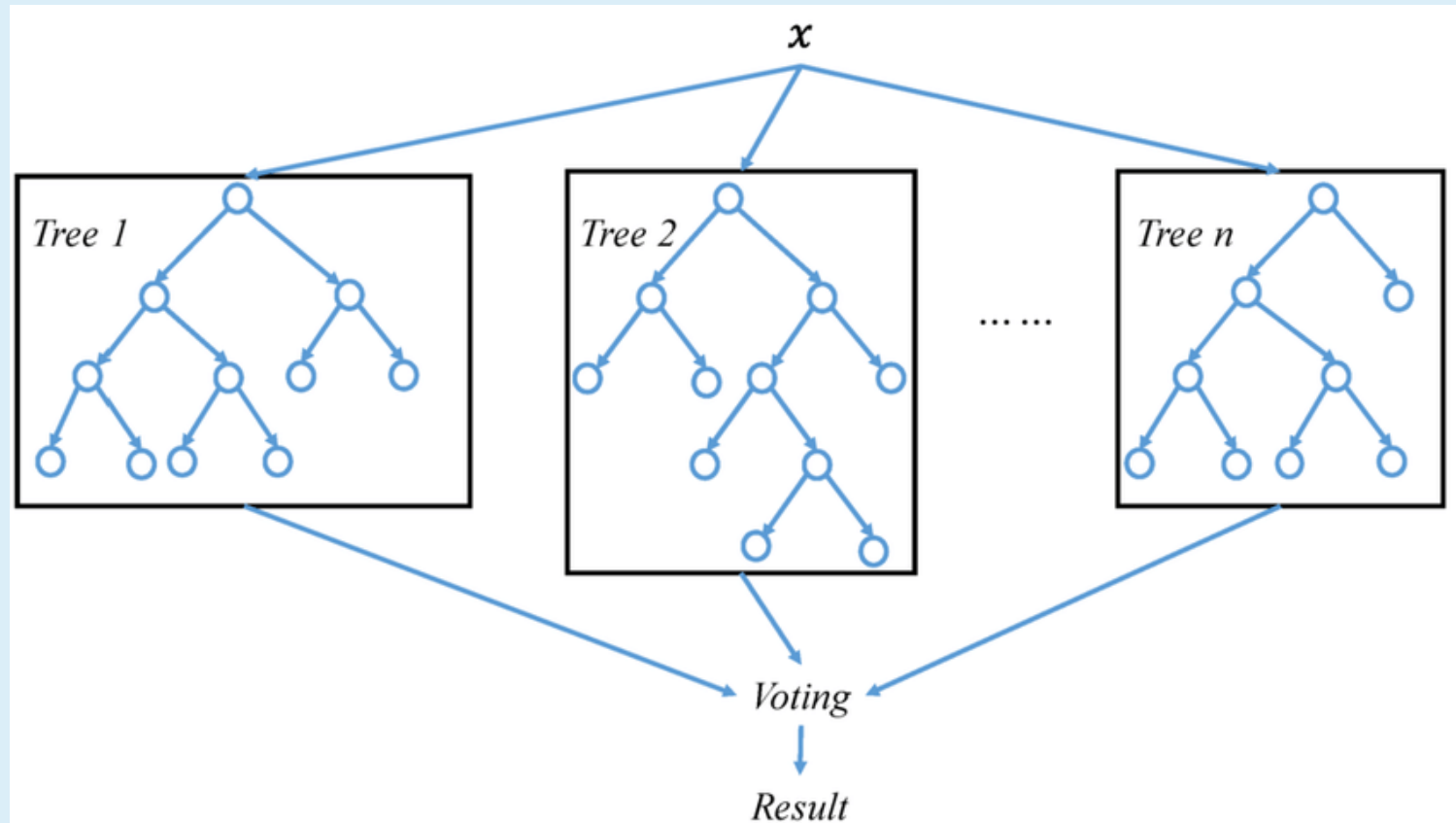
- ✓ More accurate than a single Decision Tree.
- ✓ Resistant to overfitting.

◆ Disadvantages:

- ✗ Slower than Decision Trees because many trees are
- ✗ Harder to interpret than regular Decision Trees.

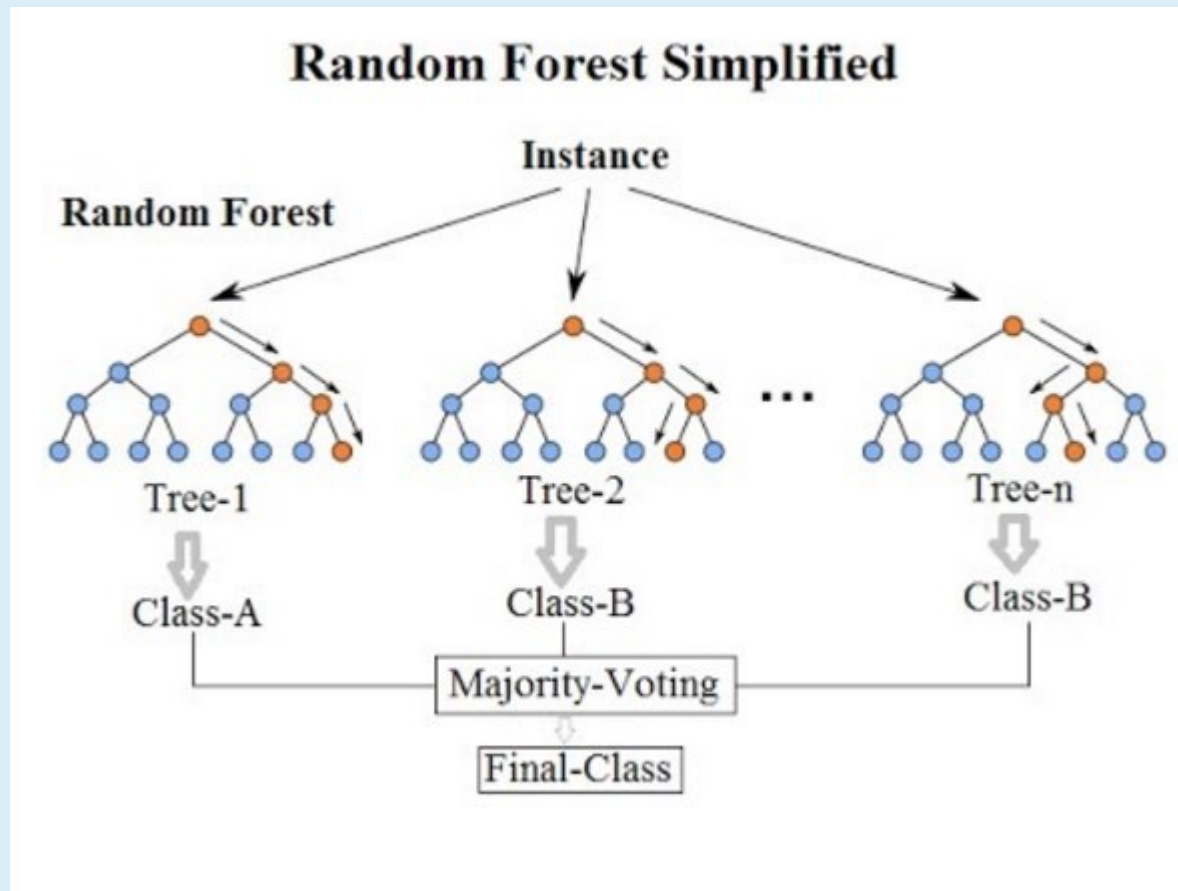
Classification Algorithm

5. Random Forest



Classification Algorithm

5. Random Forest



Classification Algorithm

5. Random Forest

Example Implementation with Python:

```
from sklearn.ensemble import RandomForestClassifier

# Initialization model Random Forest with 100 tree
rf = RandomForestClassifier(n_estimators=100)

# Train the model
rf.fit(X_train, y_train)

# Prediction
y_pred = rf.predict(X_test)
```


Comparison Method

Method	Superiority	Weakness	Suitable For
KNN	Easy to implement, no model training required	Slow for large datasets, prone to outlier	Small data with relevant features
Naive Bayes	Fast, good for text data	Feature independence assumption often inaccurate	Text classification, spam detection
SVM	Good for high-dimensional data, suitable for non-linear data	Slow for large data, difficult to choose kernel	Data with many complex features
Decision Tree	Easy to understand and visualized	Overfitting if not pruned	Small to medium data
Random Forest	More accurate, reduce overfitting	Slow, difficult to interpret	Large data with many features

Example *Classification* in Life Real

1. Face recognition on smartphones.
2. Fraud detection in financial transactions.
3. Sentiment analysis on social media.